

On the Longest Common Rigid Subsequence Problem

Nikhil Bansal · Moshe Lewenstein · Bin Ma ·
Kaizhong Zhang

Received: 16 May 2006 / Accepted: 14 February 2008 / Published online: 4 March 2008
© Springer Science+Business Media, LLC 2008

Abstract The longest common subsequence problem (LCS) and the closest substring problem (CSP) are two models for finding common patterns in strings, and have been studied extensively. Though both LCS and CSP are NP-Hard, they exhibit very different behavior with respect to polynomial time approximation algorithms. While LCS is hard to approximate within n^δ for some $\delta > 0$, CSP admits a polynomial time approximation scheme. In this paper, we study the longest common rigid subsequence problem (LCRS). This problem shares similarity with both LCS and CSP and has an important application in motif finding in biological sequences. We show that it is NP-hard to approximate LCRS within ratio n^δ , for some constant $\delta > 0$, where n is the maximum string length. We also show that it is NP-Hard to approximate LCRS within ratio $\Omega(m)$, where m is the number of strings.

Keywords Longest common subsequence · Longest common rigid subsequence · Approximation algorithms · Motif finding · Pattern matching and computational biology

Some results in this manuscript were previously presented in CPM'05 conference [14].

N. Bansal (✉)

IBM T.J. Watson Research Center, P.O. Box 218, Yorktown Heights, NY 10598, USA
e-mail: nikhil@us.ibm.com

M. Lewenstein

Department of Computer Science, Bar Ilan University, Ramat Gan 52900, Israel
e-mail: moshe@cs.biu.ac.il

B. Ma · K. Zhang

Department of Computer Science, University of Western Ontario, London, ON, N6A 5B7, Canada

B. Ma

e-mail: bma@csd.uwo.ca

K. Zhang

e-mail: kzhang@csd.uwo.ca

1 Introduction

The finding of common patterns shared by many sequences is an important computational problem that has been extensively studied in the literature. In different applications, this problem has different versions.

Suppose m sequences $s_1, s_2, \dots, s_m$ are given, where each s_i has length n . The longest common subsequence problem (LCS) seeks for a sequence s that is a subsequence of each s_i . This problem was first shown to be **NP**-hard [15] and later shown to be hard to be approximated [7]. More specifically, Jiang and Li showed in their paper [7] that there exists a constant $\delta > 0$ such that, if the longest common subsequence problem has a polynomial time approximation algorithm with performance ratio n^δ , then **P** = **NP**.

Clearly, the difficulty of LCS comes from the exponential number of possible subsequences of a given string. If the subsequence is replaced by substring and the longest common substring of all s_i is sought for, the problem can be trivially solved by examining every substring of s_1 . However, the problem of finding a string t of a given length L that matches a substring of every s_i with the fewest errors becomes **NP**-hard again under several different error measurements [9, 10]. Approximation algorithms were also studied [9–13]. Specifically, when d_H is the Hamming distance, the *closest substring problem* (CSP), which is to find a length- L string t and a substring t_i of every s_i , so that $\max_i d_H(t, t_i)$ is minimized, was studied in [12, 13]. The closest substring problem is **NP**-hard and polynomial time approximation schemes (PTAS) were provided in [12, 13]. Other error measurements were also studied [9–13].

We note that both LCS and CSP are generalizations of the trivial longest common substring problem. In LCS substrings are generalized to subsequences. While in CSP, an exact substring is generalized to be a substring with a small number of errors. It was these two generalizations that changed the trivial longest common substring problem to be hard problems, respectively.

In this paper, we study another generalization of the longest common substring problem. We call the new problem the *longest common rigid subsequence* (LCRS) and it is useful for finding motifs in bioinformatics.

Let Σ be an alphabet. Suppose $s = s[1]s[2] \cdots s[n]$ is a string over Σ . A *rigid subsequence* of s is a subsequence of s along with the relative positions of the letters. More precisely, a rigid subsequence has the form $\langle t, P \rangle$, where $t = s[i_1]s[i_2] \cdots s[i_k]$ and $P = (i_2 - i_1, i_3 - i_1, \dots, i_k - i_1)$ for $1 \leq i_1 < i_2 < \cdots < i_k \leq n$. We call P the *shape* of this rigid subsequence. A *common rigid subsequence* of several strings $s_1, \dots, s_m$ is a pair $\langle t, P \rangle$ that is a rigid subsequence of every s_i . The *longest common rigid subsequence problem* (LCRS) asks for the common rigid sequence $\langle t, P \rangle$ that maximizes the length of t .

Compared to LCS, LCRS is more restricted in the sense that the common subsequence now must appear in every s_i with the same shape. If compared with CSP, on one hand LCRS is more generalized because a shape needs to be computed; on the other hand LCRS is more restricted because it does not allow errors. Since the computational complexities of both LCS and CSP have been well-studied, it is of theoretical interest to ask about the complexity of LCRS.

In the context of computational biology and bioinformatics, LCRS has been used in the *motif* finding problem. In biochemistry, a *motif* is a recurring pattern in some DNA or protein sequences. Usually a motif is a short substring on the sequence but not all the positions on the substring are of the same importance. For example, by examining nine different protein sequences that bind to the SH3 domain, a consensus sequence, RPLXXP, was revealed as the SH3 domain-binding motif [6]. The letter X in the sequence is a wildcard that can match any amino acids, which indicates that the amino acids at the two X-positions are less important to the biochemical function of the motif. Obviously, such a motif can be represented by a common rigid subsequence, $\langle \text{RPLPP}, (1, 2, 3, 6) \rangle$.

Although different notations were used, [19] already considered a slight generalization of LCRS in the context of finding motifs. Their problem requires to find the longest common rigid subsequences that occurs in at least K of the m given sequences for a given number K . When $K = m$, this problem is exactly the LCRS problem. An exponential time algorithm was given in [19]. Later on, [17, 18] developed more efficient exponential time algorithms to the problem and variants. However, it was not shown in [17–19] whether this problem is **NP**-hard, though it was believed so.

In this paper, we answer the complexity of the LCRS problem. We begin by showing that LCRS is **Max SNP**-hard, and it cannot have an polynomial time approximation arbitrarily close to 1. Then we amplify the hardness using a natural “powering” technique and show that it is **NP**-hard to approximate LCRS within ratio $o(m)$ and n^δ for some δ , respectively. The hardness result holds even when the alphabet size is 4.

2 Hardness of the LCRS Problem

We begin by showing that the LCRS problem is **Max SNP**-hard. If a problem is **Max SNP**-hard, then there is a constant $\epsilon > 0$, such that the polynomial time approximation to that problem within ratio $1 + \epsilon$ will yield that **NP** = **P** [16]. In Sect. 2.2 we will show how to amplify this hardness to $\Omega(n^\delta)$. Our construction also implies an $\Omega(m)$ hardness, where m is the number of strings in the instance, and only uses an alphabet of size 4.

2.1 Max SNP-Hardness of LCRS

To show the **Max SNP**-hardness of LCRS, we will give a gap-preserving reduction from the maximum directed cut (Max-Dicut) problem. For more details about gap-preserving reductions, we refer the reader to [5, 16]. The Max-Dicut problem is the following.

Max-Dicut Problem Given a directed graph $G = (V, E)$, find a subset of vertices S such that the number of directed edges from S to $V \setminus S$ is maximized.

It is well known that Max-Dicut is **Max SNP**-hard, even for graphs $G = (V, E)$ with degree at most 6 [2]. We now describe the construction which reduces Max-Dicut to LCRS. The alphabet used by our construction has four letters a, b, c, and d.

Suppose the instance of Max-Dicut $G = \langle V, E \rangle$ has m vertices and n edges. In our construction, each vertex $v_i \in V$ ($i = 1, 2, \dots, m$) corresponds to a string s_i of the following form:

$$s_i = \text{xx} \overbrace{\text{ccc} \dots \text{c}}^{4(n^2+1)} \text{xx} \overbrace{\text{ccc} \dots \text{c}}^{4(n^2+2)} \text{xx} \dots \overbrace{\text{ccc} \dots \text{c}}^{4(n^2+n-1)} \text{xx}$$

where the xx are two letters to be specified later. In addition, we consider the following additional string that we call s_0 .

$$s_0 = \text{aa} \overbrace{\text{ddd} \dots \text{d}}^{4(n^2+1)} \text{aa} \overbrace{\text{ddd} \dots \text{d}}^{4(n^2+2)} \text{aa} \dots \overbrace{\text{ddd} \dots \text{d}}^{4(n^2+n-1)} \text{aa}$$

For the purpose of presentation we align $s_1, \dots, s_m$ and s_0 together as follows:

$$\begin{array}{ccccccc} s_0 : & \text{aa} & \overbrace{\text{ddd} \dots \text{d}}^{4(n^2+1)} & \text{aa} & \overbrace{\text{ddd} \dots \text{d}}^{4(n^2+2)} & \text{aa} & \dots \overbrace{\text{ddd} \dots \text{d}}^{4(n^2+n-1)} \text{aa} \\ s_1 : & \text{xx} & \text{ccc} \dots \text{c} & \text{xx} & \text{ccc} \dots \text{c} & \text{xx} & \dots \text{ccc} \dots \text{c} \text{xx} \\ \vdots & & & & & & \\ s_m : & \text{xx} & \text{ccc} \dots \text{c} & \text{xx} & \text{ccc} \dots \text{c} & \text{xx} & \dots \text{ccc} \dots \text{c} \text{xx} \end{array}$$

The letters c and d are used in our construction as delimiters. Thus each string is divided by the delimiters into n blocks of 2-letters. Each of these n blocks will correspond to an edge in E . For the block corresponding to the directed edge $\langle v_i, v_j \rangle \in E$, the two letters xx in s_i are ab, the two letters in s_j are ba, and the two letters in every other string are aa. Thus, the block corresponding to $\langle v_i, v_j \rangle$ looks as follows:

$$\begin{array}{cccc} s_0 & \dots & \text{aa} & \dots \\ \dots & \dots & \dots & \dots \\ s_{i-1} & \dots & \text{aa} & \dots \\ s_i & \dots & \text{ab} & \dots \\ s_{i+1} & \dots & \text{aa} & \dots \\ \dots & \dots & \dots & \dots \\ s_{j-1} & \dots & \text{aa} & \dots \\ s_j & \dots & \text{ba} & \dots \\ s_{j+1} & \dots & \text{aa} & \dots \\ \dots & \dots & \dots & \dots \\ s_m & \dots & \text{aa} & \dots \end{array}$$

This completes the description of the instance of LCRS. We will show that this reduction is in fact an L -reduction, which implies that the LCRS problem is **Max SNP**-hard. For more details about L -reduction, we refer the reader to [16].

Theorem 1 *The LCRS problem is Max SNP-hard.*

We begin with some simple facts.

Observation 1 Any common rigid sequence of the instance constructed above can only contain the letter $\bar{a}$.

Proof This is because s_0 does not contain the letters $\bar{b}$ and $\bar{c}$, and the sequences $s_1, \dots, s_m$ do not contain the letter $\bar{d}$. $\square$

Lemma 2 Suppose a common rigid subsequence z has length r where $r \geq 3$, and the r letters are from the blocks $k_1, k_2, \dots, k_r$ of s_0 respectively, where $(k_1 \leq k_2 \leq \dots \leq k_r)$. Then z uses letters from the blocks $k_1, k_2, \dots, k_r$ of s_i for every $i = 1, \dots, m$.

Proof Fix a string $s_i \neq s_0$, and suppose that z uses letters from the blocks $k'_1, k'_2, \dots, k'_r$ of s_i for some i . Note that if z uses two letters from the same block in s_0 , then it must also use those two letters from the same block in every other string s_i due to the rigidness property. Since $r \geq 3$, and each block has size 2, there exist two consecutive letters j and $j + 1$ of z that do not lie in the same block in s_0 , that is $k_j \neq k_{j+1}$. Let d be the distance (in s_0) between two such letters. Since each block consists of two letters, by a simple counting of letters in s_0 , we get that

$$d - 4 < \left(\sum_{k=k_j}^{k_{j+1}-1} 4(n^2 + k) \right) + 2(k_{j+1} - k_j - 1) < d$$

which by simple algebra can be rewritten as

$$d - 2 < 4n^2(k_{j+1} - k_j) + 2(k_{j+1}^2 - k_j^2) < d + 2. \quad (1)$$

Similarly, by considering the letters j and $j + 1$ of z in s_i and counting the characters, we get

$$d - 2 < 4n^2(k'_{j+1} - k'_j) + 2(k'^2_{j+1} - k'^2_j) < d + 2. \quad (2)$$

Because $k_j, k_{j+1}, k'_j, k'_{j+1}$ are all integers in the range $[1, \dots, n]$, it follows that $0 < 2k_{j+1}^2 - 2k_j^2 \leq 2n^2 - 2$. Hence by subtracting (2) from (1), it must be the case that $4n^2(k_{j+1} - k_j)$ and $4n^2(k'_{j+1} - k'_j)$ can differ by at most $(2n^2 - 2) + 3$ which is strictly less than $4n^2$. Thus it must be the case that $k'_{j+1} - k'_j = k_{j+1} - k_j$. Since $k_j \neq k_{j+1}$, dividing equations 1 and 2 throughout by $k_{j+1} - k_j$ and $k'_{j+1} - k'_j$ respectively, it follows that $2(k_{j+1} + k_j)$ and $2(k'_{j+1} + k'_j)$ can differ by at most 3. Since $k'_{j+1} - k'_j = k_{j+1} - k_j$, this can only happen if $k'_{j+1} + k'_j = k_{j+1} + k_j$. As a result, $k_j = k'_j$ and $k_{j+1} = k'_{j+1}$. $\square$

Proof of Theorem 1 Denote by I_1 an instance of Max-Dicut and I_2 be the instance of LCRS as constructed above.

The LCRS problem can be regarded as to find the maximum number of consensus columns (the columns where all the s_i have the same letter) in an ungapped multiple alignment (where extra spaces are added only at the two ends of each sequence, but not in the middle). In such a view, Lemma 2 says that if a common rigid subsequence

has more than two letters, then the corresponding blocks of all the strings are aligned together. Therefore, the difference between the numbers of leading spaces in any two aligned sequences is at most one. Thus, the sequences $s_1, \dots, s_m$ are divided into two groups S_1 and S_2 , where the sequences in S_1 have one more leading space than the sequences in S_2 . The vertices V of I_1 can be divided into two sets V_1 and V_2 correspondingly. It is easy to see that in the block corresponding to a directed edge $\langle v_i, v_j \rangle$, a consensus column exists if and only if s_i is shifted one step to the right compared to s_j , because all the other sequences always provide a letter a to the consensus column. Therefore, the number of edges in the dicut (V_1, V_2) is equal to the length of the common rigid subsequence.

On the other hand, for the optimal dicut $(V_1, V \setminus V_1)$ of I_1 , by putting one leading space at s_i for each $v_i \in V_1$, we get an ungapped multiple alignment of all the constructed sequences. Because of the above discussion, the number of consensus columns is equal to the edges in the dicut.

Thus, Max-Dicut is L -reduced to the LCRS problem which implies the **Max SNP**-hardness of LCRS. $\square$

2.2 Amplifying the Hardness

Note that in the reduction described above, we obtain a consensus column corresponding to edge $\langle v_i, v_j \rangle$ if and only if the strings s_i and s_j are aligned “properly”. The idea for amplifying the hardness would be to make this condition stronger. In particular, to obtain a consensus column we will need to align up to $O(\log n)$ strings appropriately (rather than just two strings), which we will show to be significantly harder. To do this, we define a “graph powering” procedure on a max-dicut instance.

Let $G = (V, E)$ be a directed graph with degree at most 6, let $|V| = m$ and $|E| = n$. Notice that $n = O(m)$, and recall that finding a max-dicut of G is **Max SNP**-hard. Since any directed graph has a dicut of size at least $m/4$ (by choosing a subset $S \subseteq V$ uniformly at random and derandomizing), it follows that there exist constants c and δ such that it is impossible to distinguish in polynomial time whether there is a dicut with of size at least cn or at most $(c - \delta)n$. Let k be the largest integer such that $(c/(c - \delta))^k < m$. Clearly, $k = O(\log m)$.

Let E^k denote the set $(E \times E \times \dots \times E)$ of all ordered k -tuples of directed edges in E . This set has cardinality exactly n^k . We represent an element of E^k as $e_i^k = (e_{i_1} = \langle v_{i_1}, v_{i'_1} \rangle, e_{i_2} = \langle v_{i_2}, v_{i'_2} \rangle, \dots, e_{i_k} = \langle v_{i_k}, v_{i'_k} \rangle)$. For an ordered k -tuple $e_i^k \in E^k$, let $\Pi_1(e_i^k)$ denote the ordered k -tuple of vertices $(v_{i_1}, v_{i_2}, \dots, v_{i_k})$ and let $\Pi_2(e_i^k)$ denote the ordered k -tuple of vertices $(v_{i'_1}, v_{i'_2}, \dots, v_{i'_k})$. We say that an ordered k -tuple of edges e^k is *good*, if $\Pi_1(e_i^k) \cap \Pi_2(e_i^k) = \emptyset$, i.e. the sets of vertices $\{v_{i_1}, \dots, v_{i_k}\}$ and $\{v_{i'_1}, \dots, v_{i'_k}\}$ are disjoint.

Observation 2 *There are $(1 - o(1))n^k$ good k -tuples in E^k .*

Proof If some $e^k \in E^k$ is not good, then there is some vertex v such that $v \in \Pi_1(e^k) \cap \Pi_2(e^k)$. Fix a vertex v . Since v has degree at most 6, the number of ordered k -tuples that share v is at most $(6 \times 6) \cdot k(k - 1) \cdot n^{k-2} = O(n^{k-2} \log^2 n)$. And hence the

total number of k -tuples that are not good in E^k is at most $m \cdot O(n^{k-2} \log^2 n) = O(n^{k-1} \log^2 n) = o(n^k)$. $\square$

For a good tuple e_i^k , let f_i denote the ordered pair $(\Pi_1(e_i^k), \Pi_2(e_i^k))$. We call such a pair a k -edge. Let F^k denote the set of all k -edges. By the observation 2 above, $|F^k| = (1 - o(1))n^k$.

Consider the following problem for (V, F^k) , that we call the maximum k -fold dicut problem. For a subset S of V , we say that a k -edge $f_i = (\Pi_1(e_i^k), \Pi_2(e_i^k))$ is cut by S if and only if $\Pi_1(e_i^k) \subseteq S$ and $\Pi_2(e_i^k) \subseteq V \setminus S$. The maximum k -fold dicut problem for (V, F^k) is to find a partition $(S, V \setminus S)$ of V that maximizes the number of k -edges that are cut.

The following lemma relates the max-dicut problem on G to the maximum k -fold dicut problem on the instance (V, F^k) .

Lemma 3 *The directed graph $G = (V, E)$ has a dicut of size at least r if and only if there is a k -fold dicut of size at least r^k for (V, F^k) .*

Proof Let $(S, V \setminus S)$ be a dicut of size r in G . Let $E(S)$ denote the set of the edges in this dicut. Consider the set of ordered k -tuples $E^k(S) = E(S) \times E(S) \times \cdots \times E(S)$. There are exactly r^k such ordered k -tuples in $E^k(S)$ and each of them is good, because for any $e^k \in E^k(S)$, we have that $\Pi_1(e^k) \subseteq S$ and $\Pi_2(e^k) \subseteq V \setminus S$ and hence $\Pi_1(e^k) \cap \Pi_2(e^k) = \emptyset$. Thus, there is a k -fold dicut for (V, F^k) of size least r^k .

On the other hand suppose there is a partition $(S, V \setminus S)$ that cuts r' k -edges in F^k . Let L be this set of cut k -edges. For a k -edge $f_i = (\Pi_1(e_i^k), \Pi_2(e_i^k))$, let $P_d(f_i)$ (the projection of L to its d^{th} co-ordinate) denote the edge e_{id} corresponding to the d -th coordinate of e_i^k . Let $P_d(L)$ denote the set of all edges that correspond to the d -th co-ordinate of some k -edge in L . Clearly, there is some d such that $1 \leq d \leq k$ such that $P_d(L) \geq r'^{1/k}$ (otherwise the set L cannot have cardinality r'). By definition of L , every edge in $P_d(L)$ is directed from S to $V \setminus S$, which implies that there is dicut for G of size more than $r'^{1/k}$. $\square$

This immediately implies the following:

Corollary 1 *It is impossible to distinguish in time polynomial in the size of F^k , whether there is k -fold dicut of (V, F^k) that has size at least $(cn)^k$ or at most $((c - \delta)n)^k$. Since $(c/(c - \delta))^k = \Omega(m)$, this implies an $\Omega(m)$ approximation hardness for the max k -fold dicut problem.*

Since $k = O(\log n)$, the size of the above construction is $n^{O(\log n)}$ and hence not polynomial. However we can use a standard sampling argument to show that we do not really need all possible k -tuples of E . In particular, it suffices to consider a polynomial size subset of F^k . The sampling argument is given in Lemma 5, of which the proof requires the following technical lemma.

Lemma 4 Suppose $X_1, \dots, X_n$ are 0-1 random variables, such that $\Pr[X_i = 1] = p_i$. Let $\mu = \sum_{i=1}^n p_i$ and $X = \sum_{i=1}^n X_i$. Then for any $a > 0$,

$$\Pr[X - \mu \geq a] \leq e^{-a \min(1/5, a/4\mu)}.$$

Proof The Chernoff bounds (p. 267, Corollary A.1.10 and Theorem A.1.13, [4]) state that

$$\Pr[X - \mu \geq a] \leq e^{-(a+\mu) \ln(1+a/\mu)} \quad (3)$$

and

$$\Pr[X - \mu \leq -a] \leq e^{-a^2/2\mu}.$$

Let $x = \mu/a$. Then the right-hand side of (3) can be written as $e^{-a((x+1) \ln(1+1/x) - 1)}$.

Now, $(x+1) \ln(1+1/x) - 1$ is decreasing in x . At $x = 2$, its value is $3 \ln 3/2 - 1 \geq 1/5$. For $x > 2$, it is at least $(x+1)(1/x - 1/2x^2) - 1 = 1/(2x) - 1/(2x^2) \geq 1/(4x)$. $\square$

Lemma 5 For any $\epsilon > 0$, there is a constant d and an instance $F' \subset F^k$ with size $O(n^d)$, such that with high probability, an α approximation for the maximum k -fold dicut problem on the instance (V, F') implies a $(1 + \epsilon)^2 \alpha$ approximation for the instance (V, F) .

Proof Let d be a constant greater than 1 that we will choose appropriately later. Let F' denote a random subset of F^k obtained by choosing each k -edge uniformly at random with probability $n^d/|F^k|$. Notice that F' can be obtained in polynomial time by implicitly sampling $(1 + o(1))n^d$ ordered k -tuples from E^k and rejecting the $o(1)$ fraction of these that are not good. Clearly, the expected size of F' is n^d . By a standard Chernoff bound (see Lemma 4) it follows that $\Pr[|F'| - n^d| \geq \epsilon n^d] \leq 2e^{-\epsilon^2 n^d/4}$.

Let us condition on the event that $|F'| \leq (1 + \epsilon)n^d$. Let (V_1, V_2) be a fixed partition of V , and let C denote the k -edges in F^k cut by the partition (V_1, V_2) and let $C' = F' \cap C$. Clearly, $E[C'] = |C|n^d/|F^k|$. As $|C'| \leq |F'| \leq (1 + \epsilon)n^d$, by Lemma 4 it follows that

$$\begin{aligned} \Pr\left[|C' - E[C']| \geq \epsilon \frac{n^d}{4^k}\right] &\leq 2e^{-\epsilon^2 n^{2d}/(4^{2k} \cdot 4E[C'])} < 2e^{-\epsilon^2 n^d/((1+\epsilon)4^{2k+1})} \\ &< 2e^{-\epsilon^2 n^d/4^{2k+2}}. \end{aligned} \quad (4)$$

Thus, by the union bound over all possible 2^m sets S , the statement above holds for every partition (V_1, V_2) with probability at most $2^m \cdot (2e^{-\epsilon^2 n^d/4^{2k+2}}) = O(e^{-\epsilon^2 n^{d+1}/4^{2k+2}})$. As $k = O(\log n) = O(\log m)$, we can choose d large enough such that the probability above is $o(e^{-n})$. Let x denote the size of some maximum k -fold dicut for (V, F^k) . Clearly, x is at least $|F^k|/4^k$, and hence the expected size of maximum k -fold dicut for (V, F') is at least $n^d/4^k$. Now the inequality (4) implies that with probability $1 - o(e^{-n})$ the maximum k -fold dicut for (V, F') has size in the range $[(1 - \epsilon)n^d x/|F^k|, (1 + \epsilon)n^d x/|F^k|]$, which implies the desired result. $\square$

We note that the above reduction can be derandomized to yield a polynomial time deterministic reduction using standard ideas. In particular, in the above construction we began by amplifying the max-dicut instance $k = O(\log n)$ times which gave a large gap, but the size of instance became super-polynomially large. To reduce the size back to polynomial we showed that it suffices to choose a random subset of E^k . To derandomize this step, we can use the idea of booster sets, which is an explicit collection of sets that satisfies the property of random subsets required in (4). We refer the interested reader to [3, 5] for further details and for an example where this principle is applied.

It remains to show how to reduce the maximum k -fold dicut problem to LCRS. Given an instance of (V, F') as described above, we construct an instance I of LCRS similar to that in Sect. 2.1. We have a string s_i for vertex $v_i \in V$. We have a block of two letters (separated by appropriate delimiters) for each k -edge in F' . For a k -edge $f \in F'$, if $v \in \Pi_1(f)$, we have the pattern ab corresponding to f and v . If $v \in \Pi_2(f)$, then we have the pattern ba , otherwise we have the pattern aa . Note that this instance uses $|V| + 1$ strings of length polynomial in $|F'|$.

Lemma 6 *Let $L(I)$ denote the length of the longest common rigid subsequence in I , and let $M(V, F')$ denote the size of the maximum k -fold dicut for (V, F') . Then, if $L(I) \geq 3$, we have that $L(I) = M(V, F')$.*

Proof We first show that $M(V, F') \leq L(I)$. Let $(S, V \setminus S)$ be a partition of V , and let M denote the pseudo-edges in F' that are cut (i.e. for each $f \in M$, we have that $\Pi_1(f) \subset S$ and $\Pi_2(f) \subset V \setminus S$). By shifting the strings corresponding to vertices in S by one position to the right it is easy to see that every pseudo-edge in M contributes exactly 1 to $L(I)$.

We now show that $M(V, F') \geq L(I)$. Since $L(I) \geq 3$, by Lemma 2 and the argument used in the proof of Theorem 1, any solution must consist of consensus columns and any two strings are shifted by at most 1. Let $S \subseteq V$ denote the subset of vertices corresponding to the strings in I that are shifted to the right by one step. Since only the letter a can be aligned in a consensus column, it follows that each consensus column that contributes to $L(I)$ corresponds to a pseudo-edge that is cut in $M(V, F')$. $\square$

Clearly, by Lemma 5 and 6, we obtain that

Theorem 7 *LCRS is hard to approximate within a factor of $o(m)$ and n^δ for some $\delta > 0$, unless $\mathbf{P} = \mathbf{NP}$. Here m is the number of strings and n is the length of the longest string.*

3 Discussion

Longest common rigid subsequence problem (LCRS) is similar to the longest common subsequence problem and the closest substring problem. Because the complexities of the other two problems were well studied, it is of theoretical interest to study the complexity of LCRS. We proved that approximating LCRS within ratio $o(m)$ and

n^δ for some $\delta > 0$ are **NP**-hard. Clearly, this also implies the hardness of the more general problem of finding a rigid subsequence common to k out of m input strings [17–19].

The bad worst case hardness shown here does not necessarily imply that the problem is intractable in practice. For example, when the strings are chosen uniformly at random, an algorithm with a quasi-polynomial running time of $n^{O(\log n)}$ follows easily by observing that for $m > 2 \log n$, with high probability every common rigid subsequence has length at most $O(\log n)$. We refer the reader to [14] for further details. In general, the situation in practice probably lies somewhere between worst case instances and random instances. It would be very interesting to come up with theoretical models to analyze such issues.

Motifs in biological sequences often do not require exact matches. A small number of errors are allowed in the match between the motif and the sequences. While the LCRS model of motif finding that we study here does not consider errors, the model with errors has also been studied previously [1, 8, 11, 20, 21].

Acknowledgements This research was undertaken, in part, thanks to funding from NSERC, PREA, the Canada Research Chairs Program and the German-Israel Science Foundation. We would also like to thank the anonymous referees for their numerous suggestions that improved the quality of the presentation.

References

1. Adebisi, E.F., Kaufmann, M.: Extracting common motifs under the Levenshtein measure: theory and experimentation. In: Proceedings of the Workshop on Algorithms for Bioinformatics (WABI), pp. 140–156 (2002)
2. Alimonti, P., Kann, V.: Some APX-completeness results for cubic graphs. *Theor. Comput. Sci.* **237**, 123–134 (2000)
3. Alon, N., Feige, U., Wigderson, A., Zuckerman, D.: Derandomized graph products. *Comput. Complex.* **5**(1), 60–75 (1995)
4. Alon, N., Spencer, J.: *The Probabilistic Method*. Wiley, New York (2000)
5. Arora, S., Lund, C.: In: Hochbaum, D. (ed.) *Hardness of Approximations, Approximation Algorithms for NP-hard Problems*, pp. 399–446. PWS, Boston (1996)
6. Cheadle, C., Ivashchenko, Y., South, V., Searfoss, G., French, S., Howk, R., Ricca, G., Jaye, M.: Identification of a src sh3 domain binding motif by screening a random phage display library. *J. Biol. Chem.* **269**(39), 24034–24039 (1994)
7. Jiang, T., Li, M.: On the approximation of shortest common supersequence and longest common subsequences. *SIAM J. Comput.* **24**(5), 1122–1139 (1995)
8. Keich, U., Pevzner, P.A.: Finding motifs in the twilight zone. In: Proceedings of the Sixth Annual International Conference on Computational Biology, pp. 195–204 (2002)
9. Lancot, J.K., Li, M., Ma, B., Wang, S., Zhang, L.: Distinguishing string selection problems. *Inf. Comput.* **185**(1), 41–55 (2003). Early version appeared in SODA'99
10. Li, M., Ma, B., Wang, L.: Finding similar regions in many strings. In: Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing (STOC), Atlanta, May 1999, pp. 473–482 (1999)
11. Li, M., Ma, B., Wang, L.: Finding similar regions in many sequences. *J. Comput. Syst. Sci.* **65**(1), 73–96 (2002). Early version appeared in STOC'99
12. Li, M., Ma, B., Wang, L.: On the closest string and substring problems. *J. ACM* **49**(2), 157–171 (2002). Early versions appeared in STOC'99 and CPM'00
13. Ma, B.: A polynomial time approximation scheme for the closest substring problem. In: Giancarlo, R., Sankoff, D. (eds.) *Combinatorial Pattern Matching, 11th Annual Symposium CPM, Montreal, Canada, June 21–23 2000. Lecture Notes in Computer Science*, vol. 1848, pp. 99–107. Springer, Berlin (2000)

14. Ma, B., Zhang, K.: On the longest common rigid subsequence problem. In: Apostolico, A., Crochemore, M., Park, K. (eds.) *Combinatorial Pattern Matching*, 16th Annual Symposium CPM, Jeju Island, Korea, June 19–22 2005. *Lecture Notes in Computer Science*, vol. 3537, pp. 11–20. Springer, Berlin (2005)
15. Maier, D.: The complexity of some problems on subsequences and supersequences. *J. ACM* **25**, 322–336 (1978)
16. Papadimitriou, C., Yannakakis, M.: Optimization, approximation, and complexity classes. *J. Comput. Syst. Sci.* **43**, 425–440 (1991)
17. Rajasekaran, S., Balla, S., Huang, C.: Exact algorithms for planted motif challenge problems. In: *Proceedings of the 3rd Asia Pacific Bioinformatics Conference*, pp. 249–259 (2005)
18. Rajasekaran, S., Balla, S., Huang, C., Thapar, V., Gryk, M., Maciejewski, M., Schiller, M.: Exact algorithms for motif search. In: *Proceedings of the 3rd Asia Pacific Bioinformatics Conference*, pp. 239–248 (2005)
19. Rigoutsos, I., Floratos, A.: Combinatorial pattern discovery in biological sequences: the teiresias algorithm. *Bioinformatics* **14**(1), 55–67 (1998)
20. Stormo, G., Hartzell, G.W.: Identifying protein-binding sites from unaligned DNA fragments. *Proc. Natl. Acad. Sci. USA* **86**(4), 1183–1187 (1999)
21. Waterman, M., Arratia, R., Galas, D.J.: Pattern recognition in several sequences: consensus and alignment. *Bull. Math. Biol.* **46**(4), 515–527 (1984)